prefers one shape over another, the more likely new features will be harder and harder to fit into that structure. Therefore architectures should be as shape agnostic are practical.

THE GREATER VALUE

Function or architecture? Which of these two provides the greater value? Is it more important for the software system to work, or is it more important for the software system to be easy to change?

If you ask the business managers, they'll often say that it's more important for the software system to work. Developers, in turn, often go along with this attitude. *But it's the wrong attitude.* I can prove that it is wrong with the simple logical tool of examining the extremes.

- *If you give me a program that works perfectly but is impossible to change, then it won't work when the requirements change, and I won't be able to make it work. Therefore the program will become useless.*
- *If you give me a program that does not work but is easy to change, then I can make it work, and keep it working as requirements change. Therefore the program will remain continually useful.*

You may not find this argument convincing. After all, there's no such thing as a program that is impossible to change. However, there are systems that are *practically* impossible to change, because the cost of change exceeds the benefit of change. Many systems reach that point in some of their features or configurations.

If you ask the business managers if they want to be able to make changes, they'll say that of course they do, but may then qualify their answer by noting that the current functionality is more important than any later flexibility. In contrast, if the business managers ask you for a change, and your estimated costs for that change are unaffordably high, the business managers will likely be furious that you allowed the system to get to the point where the change was impractical.

EISENHOWER'S MATRIX

Consider President Dwight D. Eisenhower's matrix of importance versus urgency (Figure 2.1). Of this matrix, Eisenhower said:

I have two kinds of problems, the urgent and the important. The urgent are not important, and the important are never urgent.[1]

Figure 2.1 Eisenhower matrix

There is a great deal of truth to this old adage. Those things that are urgent are rarely of great importance, and those things that are important are seldom of great urgency.

The first value of software—behavior—is urgent but not always particularly important.

The second value of software—architecture—is important but never particularly urgent.

Of course, some things are both urgent and important. Other things are not urgent and not important. Ultimately, we can arrange these four couplets into priorities:

1. Urgent and important
2. Not urgent and important
3. Urgent and not important
4. Not urgent and not important

Note that the architecture of the code—the important stuff—is in the top two positions of this list, whereas the behavior of the code occupies the first and *third* positions.

The mistake that business managers and developers often make is to elevate items in

position 3 to position 1. In other words, they fail to separate those features that are urgent but not important from those features that truly are urgent and important. This failure then leads to ignoring the important architecture of the system in favor of the unimportant features of the system.

The dilemma for software developers is that business managers are not equipped to evaluate the importance of architecture. *That's what software developers were hired to do.* Therefore it is the responsibility of the software development team to assert the importance of architecture over the urgency of features.

FIGHT FOR THE ARCHITECTURE

Fulfilling this responsibility means wading into a fight—or perhaps a better word is "struggle." Frankly, that's always the way these things are done. The development team has to struggle for what they believe to be best for the company, and so do the management team, and the marketing team, and the sales team, and the operations team. *It's always a struggle*.

Effective software development teams tackle that struggle head on. They unabashedly squabble with all the other stakeholders as equals. Remember, as a software developer, *you are a stakeholder*. You have a stake in the software that you need to safeguard. That's part of your role, and part of your duty. And it's a big part of why you were hired.

This challenge is doubly important if you are a software architect. Software architects are, by virtue of their job description, more focused on the structure of the system than on its features and functions. Architects create an architecture that allows those features and functions to be easily developed, easily modified, and easily extended.

Just remember: If architecture comes last, then the system will become ever more costly to develop, and eventually change will become practically impossible for part or all of the system. If that is allowed to happen, it means the software development team did not fight hard enough for what they knew was necessary.

1. From a speech at Northwestern University in 1954.

II

STARTING WITH THE BRICKS: PROGRAMMING PARADIGMS

Software architecture begins with the code—and so we will begin our discussion of architecture by looking at what we've learned about code since code was first written.

In 1938, Alan Turing laid the foundations of what was to become computer programming. He was not the first to conceive of a programmable machine, but he was the first to understand that programs were simply data. By 1945, Turing was writing real programs on real computers in code that we would recognize (if we squinted enough). Those programs used loops, branches, assignment, subroutines, stacks, and other familiar structures. Turing's language was binary.

Since those days, a number of revolutions in programming have occurred. One revolution with which we are all very familiar is the revolution of languages. First, in the late 1940s, came assemblers. These "languages" relieved the programmers of the drudgery of translating their programs into binary. In 1951, Grace Hopper invented A0, the first compiler. In fact, she coined the term *compiler*. Fortran was invented in 1953 (the year after I was born). What followed was an unceasing flood of new programming languages—COBOL, PL/1, SNOBOL, C, Pascal, C++, Java, ad infinitum.

Another, probably more significant, revolution was in programming *paradigms*. Paradigms are ways of programming, relatively unrelated to languages. A paradigm tells you which programming structures to use, and when to use them. To date, there have been three such paradigms. For reasons we shall discuss later, there are unlikely

to be any others.

3
PARADIGM OVERVIEW

The three paradigms included in this overview chapter are structured programming, object-orient programming, and functional programming.

STRUCTURED PROGRAMMING

The first paradigm to be adopted (but not the first to be invented) was structured programming, which was discovered by Edsger Wybe Dijkstra in 1968. Dijkstra showed that the use of unrestrained jumps (`goto` statements) is harmful to program structure. As we'll see in the chapters that follow, he replaced those jumps with the more familiar `if/then/else` and `do/while/until` constructs.

We can summarize the structured programming paradigm as follows: